

What did we learn so far?

— DFA: Deterministic finite automaton

- Limited memory

- Deterministic behaviour.

— NFA: Nondeterministic finite automaton

- Limited memory

- Nondeterministic behaviour.

However, $NFA = DFA$

Any language that is recognized by a DFA/NFA is called a regular language.

Regular Expressions.

In simple terms, regular expressions describe languages.

Basically, it's a notation for describing strings in a set.

we use regular operations to build more complicated expressions.

— regular operations

- Union ($\cup$)
- Concatenation ($\circ$)
- Kleen star ($*$)

$$\text{Ex: } (0 \cup 1) 0^* \equiv (\{0\} \cup \{1\}) \circ \{0\}^* \\ (\{0, 1\}) \circ \{0\}^*$$

If Σ is the **alphabet**, the regular expression Σ describes the language consisting of all length 1 strings over the alphabet Σ .

$$\Sigma = \{0, 1\}$$

$$\Sigma \equiv (0 \cup 1) \equiv \{0\} \cup \{1\} \\ \equiv \{0, 1\}$$

↑
using Σ as a
regular
expression.

Σ^* describes the language consisting of all strings over the alphabet Σ .

$$\Sigma = \{0, 1, 2\}$$

$$\Sigma^* = (0 \cup 1 \cup 2)^* = (\{0\} \cup \{1\} \cup \{2\})^*$$

$$= \{ \varepsilon, 0, 1, 2, 00, 000, 0110, \dots \}$$

In regular expressions star operation are done first, followed by concatenation, and finally, union, unless parenthesis change the usual order.

1. Paranthesis
2. Star
3. Concatenation
4. Union.

Def : Regular Expressions

R is a regular expression,

if R is,

1. a , or some $a \in \Sigma$ $\{a\}$

2. ϵ  $\{\epsilon\}$

3. ϕ  $\{\epsilon\}$

 these are not equal

4. $R_1 \cup R_2$, where R_1 and R_2 are regular expressions

5. $R_1 \circ R_2$, where R_1 and R_2 are regular expressions

6. R_1^* , where R_1 is a regular expression.

$$\text{Ex: } R \cup \emptyset = R$$

$$\Sigma = \{0, 1\}$$

$$R = 1$$

$$L(R) = \{1\}$$

$$L(R \cup \emptyset) = \{1\} \cup \{\} = \{1\}$$

$$R \cup \epsilon \neq R \quad (\text{unless } R = \emptyset)$$

$$R = 1$$

$$L(R) = \{1\}$$

$$L(R \cup \epsilon) = \{1\} \cup \{\epsilon\} = \{1, \epsilon\}$$

Remember: Regular expressions describe languages

$$0^* 1 0^* = \{w \mid w \text{ contains a single } 1\}$$

Simple exercise; let $\Sigma = \{0, 1\}$

what are the languages described by the following regular expressions?

1. $(01)^* = \{w \mid w \text{ contains } n \text{ copies of } 01 \text{ where } n \geq 0\}$

2. $1^* \circ \varepsilon = 1^*$

3. $0^* \circ \phi = 0^* \circ \{\} = \phi$

4. $(01)^* \cup \varepsilon = (01)^*$

5. $(0 \cup \varepsilon) 1^* = \underline{0} 1^* \cup 1^*$

6. $(0 \cup \varepsilon) \circ (1 \cup \varepsilon) = \{0, \varepsilon\} \circ \{1, \varepsilon\} = \{01, 0, 1, \varepsilon\}$

7. $\phi^* = \{\}^* = \{\varepsilon\} = \varepsilon$

Regular Expressions

=

Regular Languages.

Any regular language (a language that is recognized by a DFA/NFA) can be expressed by a regular expression.

Theorem 1.54

A language is regular iff
some regular expression describes
it.

Let A be a language,

A is regular $\iff$ some regular expression
describes A .

"If part" $\Leftarrow$ (Some regular expression describes $A \Rightarrow A$ is regular)

Lemma 1.55

If a language A is described by a
regular expression, then A is regular.

Proof: Let R be a regular expression
then we will convert it to an NFA.

$$L(R) = A$$

1. $R = a, a \in \Sigma, L(R) = \{a\}$

NFA: $\rightarrow \textcircled{1} \xrightarrow{a} \textcircled{2}$

2. $R = \epsilon, L(R) = \{\epsilon\}$

$\rightarrow \textcircled{1}$

3. $R = \phi, L(R) = \{\}$

$\rightarrow \textcircled{1}$

4. $R = R_1 \cup R_2$

5. $R = R_1 \circ R_2$

6. $R = R_1^*$

} use the already created
NFA's of R_1, R_2 then
create larger NFA's
by following Thm 1.45, 1.47,
1.49

Ex: $(a \circ b \cup a)^*$

$\Sigma = \{a, b\}$

$a \rightarrow \text{state} \xrightarrow{a} \text{final state}$

$b \rightarrow \text{state} \xrightarrow{b} \text{final state}$

$a \circ b \rightarrow \text{state} \xrightarrow{a} \text{state} \xrightarrow{\epsilon} \text{state} \xrightarrow{b} \text{final state}$

$a \circ b \cup a \rightarrow$

$(a \circ b \cup a)^* \rightarrow$

"only if" part $\Rightarrow$

Lemma 1.60

If a language is regular,
then it is described by a
regular expression.

Idea!

Step 1: DFA $\longrightarrow$ GNFA

$\swarrow$ new type of
finite automaton

Step 2: GNFA $\longrightarrow$ regular expression

Generalized NFA is simply a
Nondeterministic finite automaton wherein
the transition arrows may have any
regular expression as labels, instead
of only one symbol or ϵ .



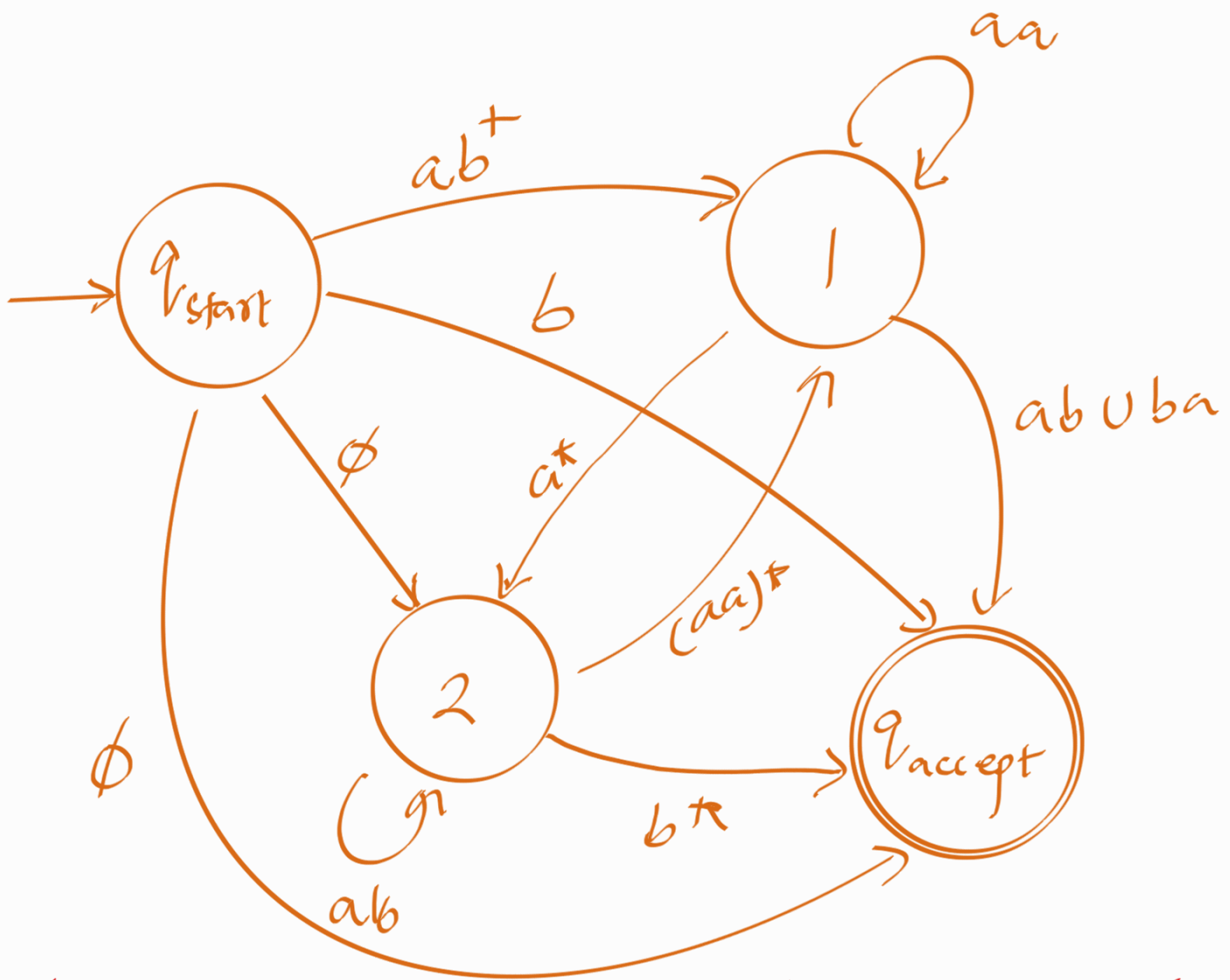
we want the DFA to convert into a
GNFA of above pattern.

GNFA follows a special set of conditions.

following conditions needs to be met if we want to call a machine GNFA.

- * Start state has transition arrows going to every other state, but no arrows coming in from any state.
- * There is only a single accept state, and it has arrows coming in from every other state but no arrows going to any other state.
- * Accept state cannot be the start state.
- * Except for start state and accept state, one arrow goes from every state to every other state and also from each state to itself.

Example GNFA

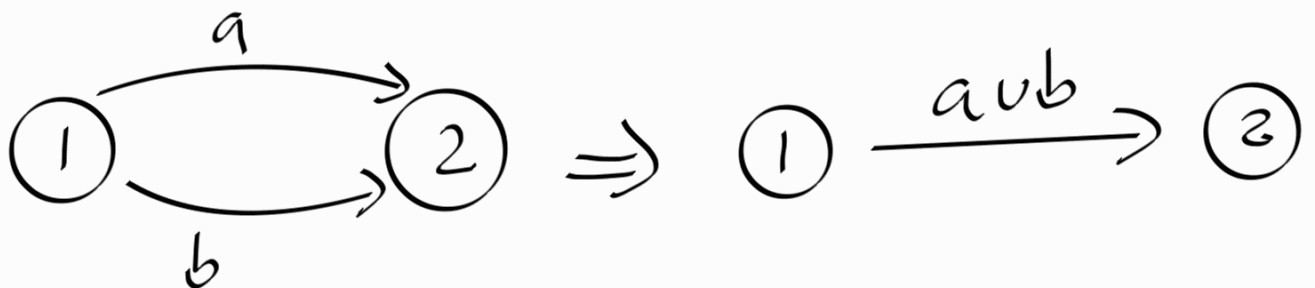


Note that ϕ transition means that there is no direct transition between the two states.

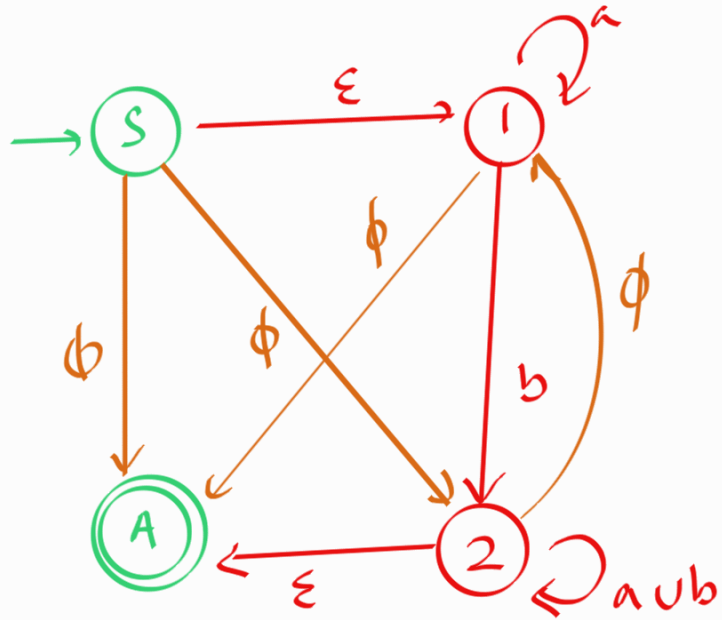
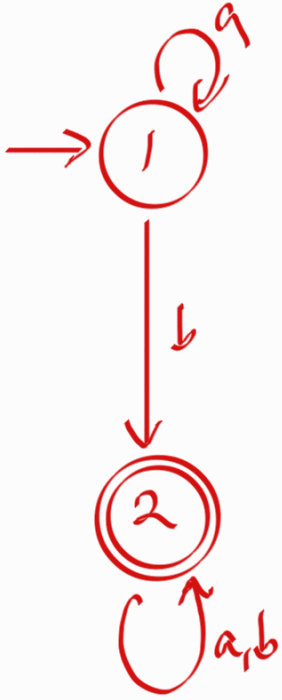
Let's look at how we can transform a DFA to a GNFA.

1. Add a new start state and add ϵ transitions to the old start state.
2. Add a new accept state and put ϵ -transitions to new accept state from all old accept states.
3. If any arrows have multiple labels (or if there are multiple arrows going between two states in the same direction), we replace each with a single arrow whose label is the union of the previous labels.

ex:



Let's look at a simple DFA and convert it to a GNFA.



Now, we need to convert the GNFA into a regular expression.

We systematically reduce the number of states of the given GNFA one by one until it has 2 states.

- Starting state
- Accept state
- Arrows between these 2 states with the final regular expression.

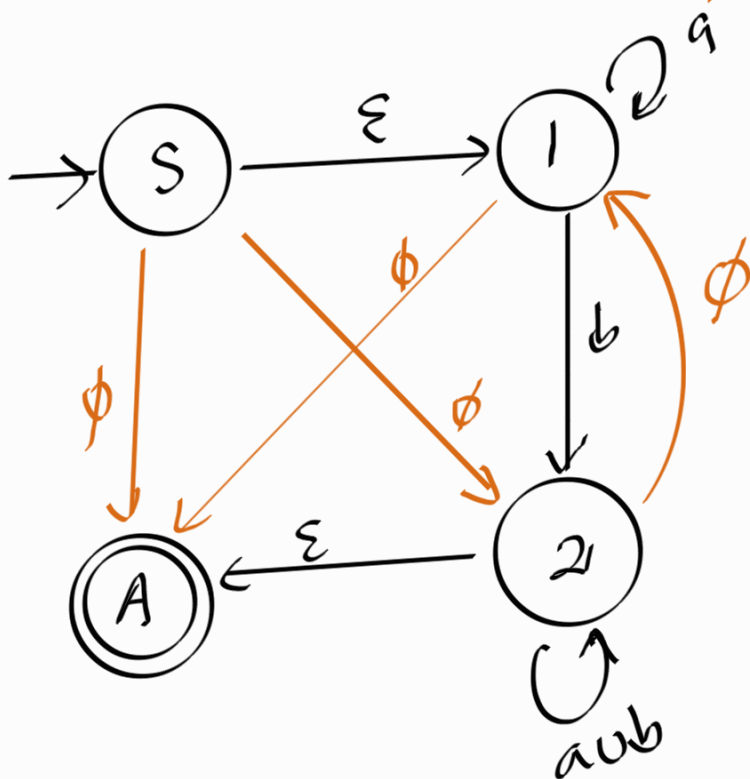
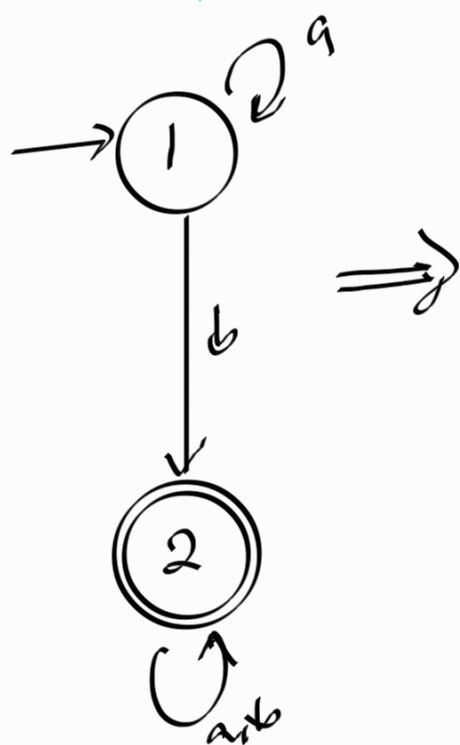
* The key idea about reducing the number of states is called Ripping.

Basically, we rip a state and repair the damage, so that the same language is recognized by the GNFA.

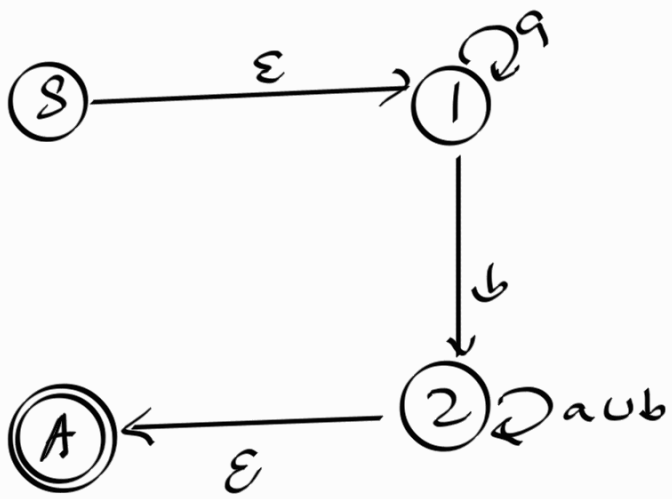
we can rip any state except start
and accept state.

we have to consider all the paths
that can go through the rip state
that we consider and repair them.

Example: DFA $\rightarrow$ GNFA $\rightarrow$ Regular Expression

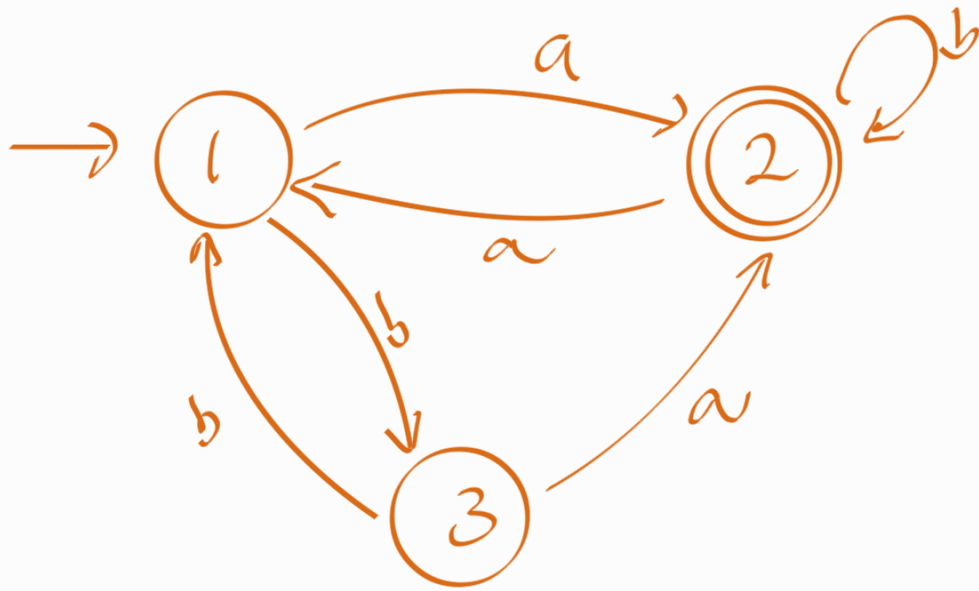


Sometimes we do not include the unnecessary ϕ transitions as it clutters the figure.



ripping ①

Another Example.



Let's try to convert to a GNFA

